LINUX / SYSTEMS

Troubleshooting Methodology Visual Guide

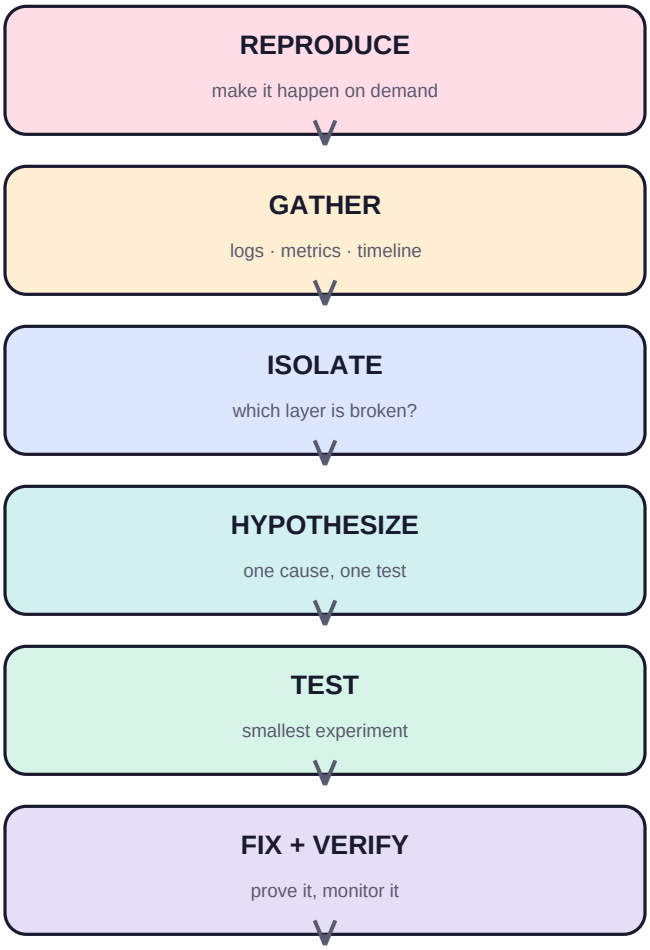
Reproduce, isolate layers, test hypotheses
— a systematic flow for any outage.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Flowchart

The same loop works for a dead server or a flaky script.



The loop matters more than the tools: skipping REPRODUCE means you 'fix' things you can't prove were broken.

STEP 2/6

Reproduce: Define Broken Precisely

Vague symptoms get vague fixes.

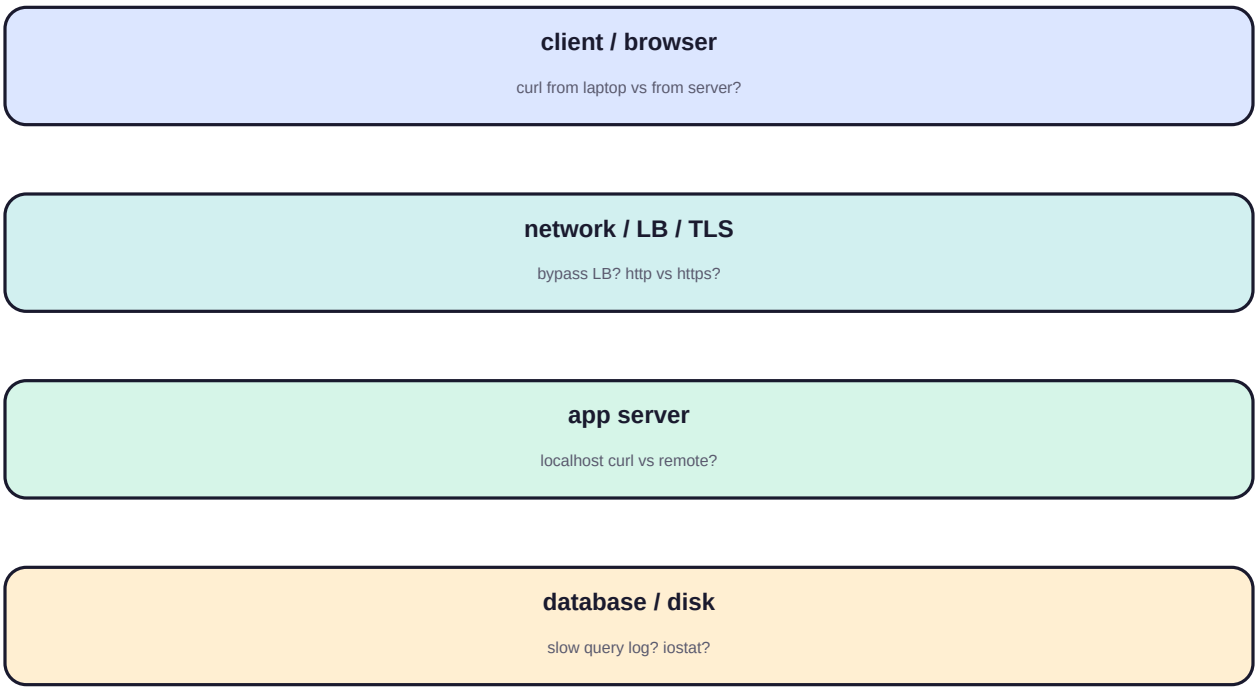
Vague	Precise (reproducible)
'the site is down'	'curl https://x/ returns 502 in 300ms, eve
'it's slow sometimes'	'p99 latency 4s on /search, 14:00–15:00 da
'deploy broke it'	'v2.4.1: /health 500s; v2.4.0 fine; rollba

- Write the repro as a COMMAND someone else can run — that's your before/after proof.
- Note what CHANGED recently: deploys, config, certs, traffic, cron. Most outages have a birthday.
- If you can't reproduce it, you can't verify the fix — say so out loud before 'fixing'.

STEP 3/6

Isolate: Bisect the Stack

Split the system in half; test which half is guilty.



Each question eliminates HALF the stack. Three questions → one suspect layer.

```
$ curl localhost:8080/health     # app itself OK? (cuts network+LB)
$ curl https://app/health        # now add TLS+LB back
# works locally, fails remotely → the fault is BETWEEN, not IN
```

STEP 4/6

Hypothesize & Test: One Variable

A hypothesis you can't test is a horoscope.

Weak	Strong (testable)
'maybe DNS'	'dig fails for this name on this host, wor
'probably the firewall'	'tcpdump shows SYN, no SYN-ACK on port 543
'the deploy'	'rollback to v2.4.0 clears the 500s in 60

- State the expected result BEFORE running the test — otherwise every result 'confirms' your bias.
- Cheapest test first: a 10-second curl beats a 30-minute config rewrite.
- One change per test. Two changes + success = you learned nothing durable.

STEP 5/6

Evidence Tools by Layer

Reach for the right lens per layer.

Layer	Tools	Shows
app	logs, journalctl	errors, stack traces
process	ps, strace, lsof	what it's doing / holding
system	dmesg, vmstat, iostat	kernel, memory, disk truth
network	ss, tcpdump, dig	sockets, packets, DNS
time	timedatectl, ntp	clock skew breaks TLS/auth

Clock skew is the silent killer: expired-looking certs, Kerberos failures, and 'random' auth errors. Check the clock early — it costs five seconds.

STEP 6/6

After the Fix: Close the Loop

An unverified fix is a rumor.

- ✓ Re-run the repro command — confirm the precise symptom is gone
- ✓ Watch for 15+ minutes (or one full cycle) — flaky fixes resurface
- ✓ Check you didn't break the neighbors — run the adjacent repros too
- ✓ Write it down: symptom, cause, fix, time — your future on-call self says thanks
- ✓ Add the monitor/alert that would have caught it sooner
- ✓ Blameless note: what in the SYSTEM allowed this, not who

STEP ✓/✓

Key Takeaways

Troubleshooting Methodology

- Loop: reproduce → gather → isolate → hypothesize → test → fix → verify. Don't skip steps.
- Define 'broken' as a runnable command — vague symptoms get vague fixes.
- Bisect the stack: each test should eliminate half the suspects.
- Hypotheses must be testable with a stated expected result; cheapest test first.
- Close the loop: verify, monitor, document, and add the alert that would have caught it.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev